PROJECT REPORT

Smart Library Seat Reservation & Study Room Booking System

A Backend Application using Spring Boot, Java & MySQL

Submitted By:

Uchit Yadav (23BCS10465)
Kartik Kashyap (23BCS12545)

Under the Guidance of:
Er. Rahul Durgapal
Project Supervisor

BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE & ENGINEERING

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

Academic Year: 2025 – 2026

BONAFIDE CERTIFICATE

This is to certify that the project report entitled

"Smart Library Seat Reservation & Study Room Booking System"

is a Bonafide work done by

Uchit Yadav (23BCS10465) & Kartik Kashyap (23BCS12545)

students of B.E (Computer Science & Engineering), Chandigarh University. The project work was carried out under our supervision during the Academic Year 2025 – 2026.

Sanjeev Kumar HEAD OF DEPARTMENT Dept. of CSE Chandigarh University	**Er. Rahul Durgapal** PROJECT SUPERVISOR Dept. of CSE Chandigarh University

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to all those who contributed to the successful completion of this project.

We are deeply thankful to Er. Rahul Durgapal, our Project Supervisor, Department of Computer Science & Engineering, Chandigarh University, for his invaluable guidance, continuous support, and constructive feedback throughout the development of this project. His technical insights and encouragement were instrumental in shaping this work.

We extend our gratitude to Prof. Sanjeev Kumar, Head of Department, CSE, Chandigarh University, for providing the necessary academic infrastructure and for his continuous motivation.

We also thank the faculty members of the Department of Computer Science & Engineering at Chandigarh University for their academic support and for creating an environment that fosters learning and innovation.

Finally, we are grateful to our families and friends for their constant encouragement and moral support throughout this journey.

Uchit Yadav & Kartik Kashyap
B.E CSE, Chandigarh University

TABLE OF CONTENTS

Section	Title	Page
—	Abstract	i
—	List of Figures	ii
—	List of Tables	iii
—	Abbreviations & Symbols	iv
Chapter 1	Introduction	1
1.1	Problem Identification	1
1.2	Task Identification	2
1.3	Organization of Report	2
Chapter 2	Literature Survey	3
2.1	Existing Systems Comparison	3
2.2	Problem Definition & Objectives	4
Chapter 3	Design Flow & Process	5
3.1	Design Constraints	5
3.2	Alternative Design 1 — Monolithic REST API	6
3.3	Alternative Design 2 — Microservices Architecture	6
3.4	Best Design Selection	7
3.5	System Architecture & Data Flow	7
3.6	Database Design	9
Chapter 4	Results, Analysis & Validation	12
4.1	API Testing Results	12
4.2	Conflict Detection Validation	13
4.3	Security Testing	14
Chapter 5	Conclusion & Future Work	15
—	References	17
—	Appendix — User Manual	18

LIST OF FIGURES

Figure No.	Title	Page
Figure 3.1	System Architecture — Layered (N-Tier) Overview	7
Figure 3.2	Data Flow Diagram — Seat Booking Request Lifecycle	8
Figure 3.3	Entity Relationship Diagram (ERD)	9
Figure 3.4	Database Table Relationship Diagram	11
Figure 4.1	API Testing Flow — Authentication Sequence	12
Figure 4.2	Overlap Detection Logic Diagram	13

LIST OF TABLES

Table No.	Title	Page
Table 1.1	Project Timeline	2
Table 2.1	Comparison of Existing Library Management Systems	3
Table 3.1	Design Constraints Summary	5
Table 3.2	Alternative Design Comparison	7
Table 3.3	Users Table Schema	9
Table 3.4	Seats Table Schema	10
Table 3.5	Bookings Table Schema	11
Table 4.1	API Test Cases and Results	12
Table 4.2	Role-Based Access Control Test Results	14

ABBREVIATIONS & SYMBOLS

Abbreviation	Full Form
API	Application Programming Interface
REST	Representational State Transfer
JWT	JSON Web Token
JPA	Java Persistence API
ORM	Object Relational Mapping
RBAC	Role-Based Access Control
DTO	Data Transfer Object
BCrypt	Blowfish Crypt — Password Hashing Algorithm
SQL	Structured Query Language
HTTP	HyperText Transfer Protocol
MVC	Model View Controller
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
CRUD	Create, Read, Update, Delete
CSE	Computer Science & Engineering
UI	User Interface
AI	Artificial Intelligence

ABSTRACT

This project presents a Smart Library Seat Reservation and Study Room Booking System — a web-based backend application developed to automate and digitally manage library seat allocation in academic institutions. The system allows students to check seat availability in real time, make reservations online, and manage their bookings through a secure RESTful API.

The backend is built using Spring Boot 3.2 (Java 17), MySQL 8, Spring Data JPA with Hibernate, and JSON Web Token (JWT) for stateless authentication and authorization. The system supports two user roles — USER and ADMIN — with clearly defined and enforced permissions at every API endpoint.

A companion frontend web application has also been developed using React 18, providing a fully functional user interface for students and administrators. The frontend was built with assistance from AI-based development tools and is designed to integrate directly with the Spring Boot REST API backend.

This document covers the project objectives, literature survey of existing systems, system architecture, alternative design approaches, database design with entity-relationship diagrams, API structure, request lifecycle, results and testing validation, and future scope of the application.

Keywords: Spring Boot, JWT, REST API, MySQL, JPA, Hibernate, Role-Based Access Control, Double-Booking Prevention, Library Management System.

CHAPTER 1

INTRODUCTION

1.1 Problem Identification

In most colleges and universities, students face significant difficulty finding available seats in the library, particularly during examination periods. The traditional approach of manual seat allocation using physical registers is time-consuming, inefficient, and provides no real-time visibility into seat availability. This leads to wasted time for students, overcrowding in popular sections, and severe underutilization of dedicated study rooms.

The absence of a digital system also creates administrative burden. Library staff must manually track bookings, resolve conflicts when two students claim the same seat, and generate reports on space utilization — tasks that could be fully automated with a suitable software solution.

This project addresses that problem by providing a digital, backend-driven platform where students can view and reserve library seats online through a RESTful API. The system eliminates physical seat registers, enables real-time availability checks, and provides administrators with a centralized management interface.

1.2 Task Identification

The following primary tasks were identified for the development of this system:

- Design and implement a secure RESTful API using Spring Boot 3.2 and Java 17.
- Create a normalized relational database schema in MySQL 8 using JPA entities.
- Implement JWT-based stateless authentication and role-based access control.
- Develop conflict detection logic to prevent double booking of seats.
- Build an Admin module for centralized management of seats, users, and bookings.
- Develop a companion React 18 frontend with AI assistance for user interaction.
- Test all major API flows including authentication, booking, and access control.

1.3 Project Timeline

Phase	Activity	Duration
Phase 1	Project Setup, Maven configuration, DB connection	Week 1
Phase 2	Entity design, JPA mapping, Schema generation	Week 2
Phase 3	Repository layer and custom queries	Week 3
Phase 4	Service layer and business logic	Weeks 4–5
Phase 5	Security layer — JWT + Spring Security	Week 6
Phase 6	Controller layer and REST endpoint exposure	Week 7
Phase 7	Frontend development (React + AI assistance)	Week 8
Phase 8	Integration testing and report preparation	Weeks 9–10

Table 1.1: Project Timeline

1.4 Organization of the Report

The rest of this report is organized as follows: Chapter 2 presents a comparative literature survey of existing library management systems and related work. Chapter 3 covers the design process including constraints, alternative architectures, the final selected design, system architecture diagrams, data flow diagrams, and database schema. Chapter 4 presents the testing results and validation. Chapter 5 concludes the report and outlines future enhancements.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview

Several commercial and institutional systems have been developed over the years to manage library resources and seating. A review of these existing systems reveals the gaps that the Smart Library Seat Reservation System aims to address. The following comparison examines the most widely referenced tools in this domain.

2.2 Comparison of Existing Systems

System / Tool	Key Features	Limitations
LibSeat (Open Source)	Basic seat reservation, web interface, admin panel	No JWT auth, no role-based access, limited conflict detection
BookMySlot (Commercial)	Time-slot based booking, email notifications	Paid/proprietary, no open API, not customizable for institutions
LibCal by Springshare	Room and seat booking, calendar integration, analytics	Expensive SaaS, requires institutional subscription, no offline support
KOHA Library System	Full library management — cataloguing, circulation, seat booking	Complex setup, heavy for small institutions, no REST API by default
Custom PHP Systems (Various Institutions)	Basic reservation forms, manual admin verification	No real-time updates, frequent double-booking, poor scalability
Our System (This Project)	JWT auth, RBAC, conflict detection, REST API, Spring Boot backend, React frontend	No mobile app yet, email notification planned for future release

Table 2.1: Comparison of Existing Library Management Systems

The survey clearly shows that while several solutions exist, most are either commercially prohibitive, overly complex for academic deployments, or lack critical technical features such

as stateless JWT authentication, programmatic REST APIs, and robust conflict detection. Our system is purpose-built for academic institutions and addresses each of these gaps directly.

2.3 Problem Definition

Based on the literature review, the core problem is defined as follows: Academic institutions require a lightweight, secure, and scalable backend system that enables students to reserve library seats in real time, prevents booking conflicts, and provides administrators with centralized control — all exposed through a standard REST API that can be integrated with any frontend or mobile application.

2.4 Goals and Objectives

- Develop a production-grade Spring Boot REST API for seat reservation and study room booking.
- Implement JWT-based authentication with role-based access control (USER and ADMIN roles).
- Design a normalized MySQL database with proper foreign key relationships.
- Implement overlap detection logic to prevent double-booking of any seat.
- Provide a centralized Admin dashboard for system-wide management.
- Ensure the API can integrate with any frontend — React (already built), Angular, Flutter, or others.

CHAPTER 3

DESIGN FLOW & PROCESS

3.1 Design Constraints

The following constraints were identified and considered throughout the design and development of the system:

Constraint Type	Description
Economic	Must use open-source technologies only — Spring Boot, MySQL, Java — to eliminate licensing costs for academic institutions.
Environmental	Designed as a cloud-deployable application; minimal hardware requirements with an embedded Tomcat server.
Security	All endpoints secured with JWT tokens. Passwords stored using BCrypt hashing. Role-based access enforced at the API level.
Performance	Pagination required on all list endpoints. FetchType.LAZY used on all JPA relationships to prevent N+1 query issues.
Ethical	User data is protected — passwords are never stored in plain text. Admin cannot access user passwords.
Maintainability	Layered architecture (Controller, Service, Repository) with DTOs ensures each layer is independently testable.
Scalability	Stateless JWT authentication means the system can be horizontally scaled without session management issues.
Interoperability	Pure JSON REST API — no frontend coupling. Frontend built with React (AI-assisted) connects via standard HTTP calls.

Table 3.1: Design Constraints Summary

3.2 Alternative Design 1 — Monolithic REST API Architecture

In this design, all modules (Authentication, Seat Management, Booking, Admin) reside within a single Spring Boot application. Each module is a separate package, but they all run as one deployable unit and share the same database.

Advantages: Simple to develop, easy to test locally, single deployment artifact, low infrastructure overhead. Ideal for an academic project with a defined and bounded scope.

Disadvantages: As the application grows, the single codebase can become difficult to scale independently. For example, if the Booking module receives much higher load than the Admin module, there is no way to scale just that module without scaling the entire application.

3.3 Alternative Design 2 — Microservices Architecture

In this design, each module (Auth Service, Seat Service, Booking Service, Admin Service) is deployed as a separate Spring Boot application with its own database. An API Gateway routes requests to the appropriate service, and services communicate via REST or message queues (e.g., RabbitMQ).

Advantages: Each service can be independently scaled, deployed, and maintained. Technology heterogeneity is possible. Highly resilient.

Disadvantages: Significantly higher complexity — requires API Gateway, service discovery (Eureka), distributed tracing, inter-service authentication, and network latency management. Overkill for the current scope and student user base size.

3.4 Best Design Selection and Justification

Criterion	Monolithic REST API	Microservices
Development Complexity	Low — single project, simple setup	High — multiple services, infra needed
Deployment	Single JAR file, embedded Tomcat	Multiple containers, Docker/K8s required
Testing	Easy unit + integration tests	Complex distributed testing
Scalability	Vertical scaling (adequate for scope)	Horizontal per-service (overkill)
Maintainability	Good with layered architecture	Excellent but costly to manage
Suitable for this Project	YES — recommended	No — excessive overhead

Table 3.2: Alternative Design Comparison

Decision: The Monolithic REST API architecture was selected. The system scope — a single institution, bounded number of users, and clearly defined modules — does not warrant microservices complexity. The layered architecture within the monolith provides all the separation of concerns, testability, and maintainability required while remaining practical to develop and deploy.

3.5 System Architecture

The application follows a Layered (N-Tier) Architecture. The diagram below illustrates how each layer communicates exclusively with the layer directly below it, maintaining strict separation of concerns.

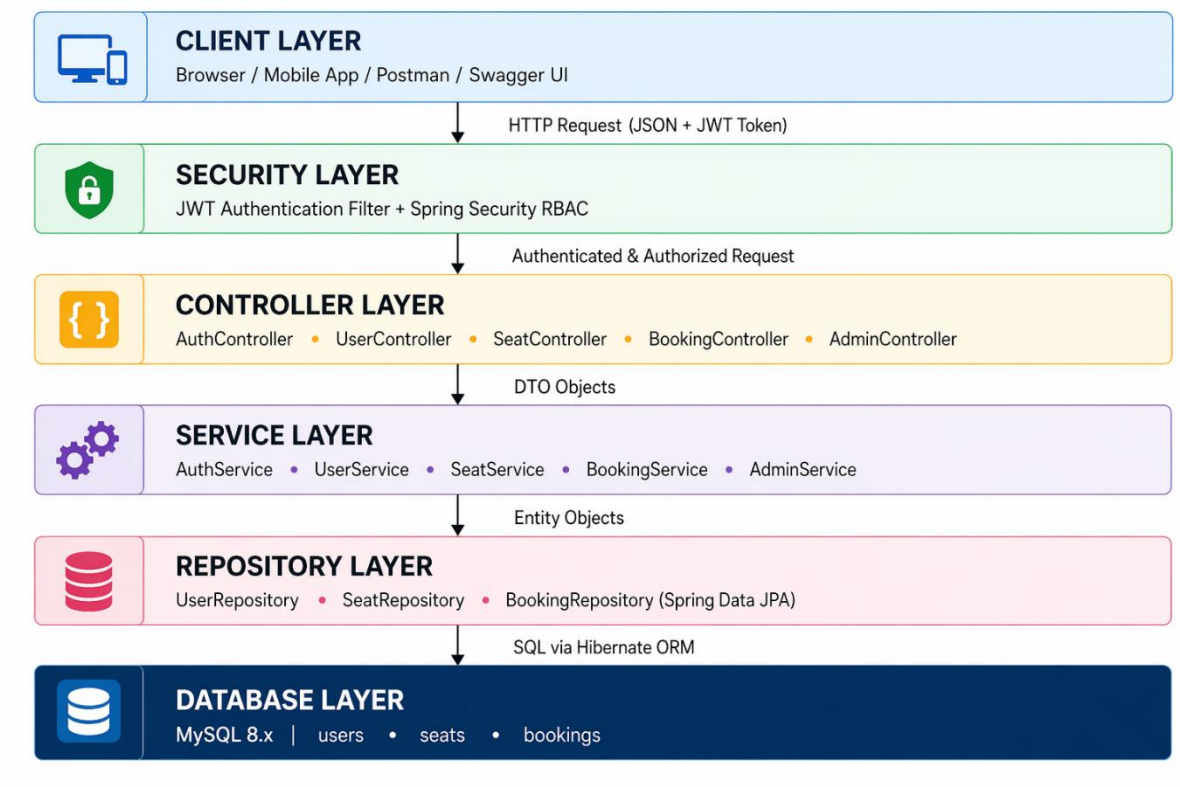

Figure 3.1: System Architecture — Layered (N-Tier) Overview

3.6 Data Flow Diagram — Seat Booking Request

The following diagram illustrates the complete data flow for a typical seat booking request, from the client through every layer of the application to the database and back.

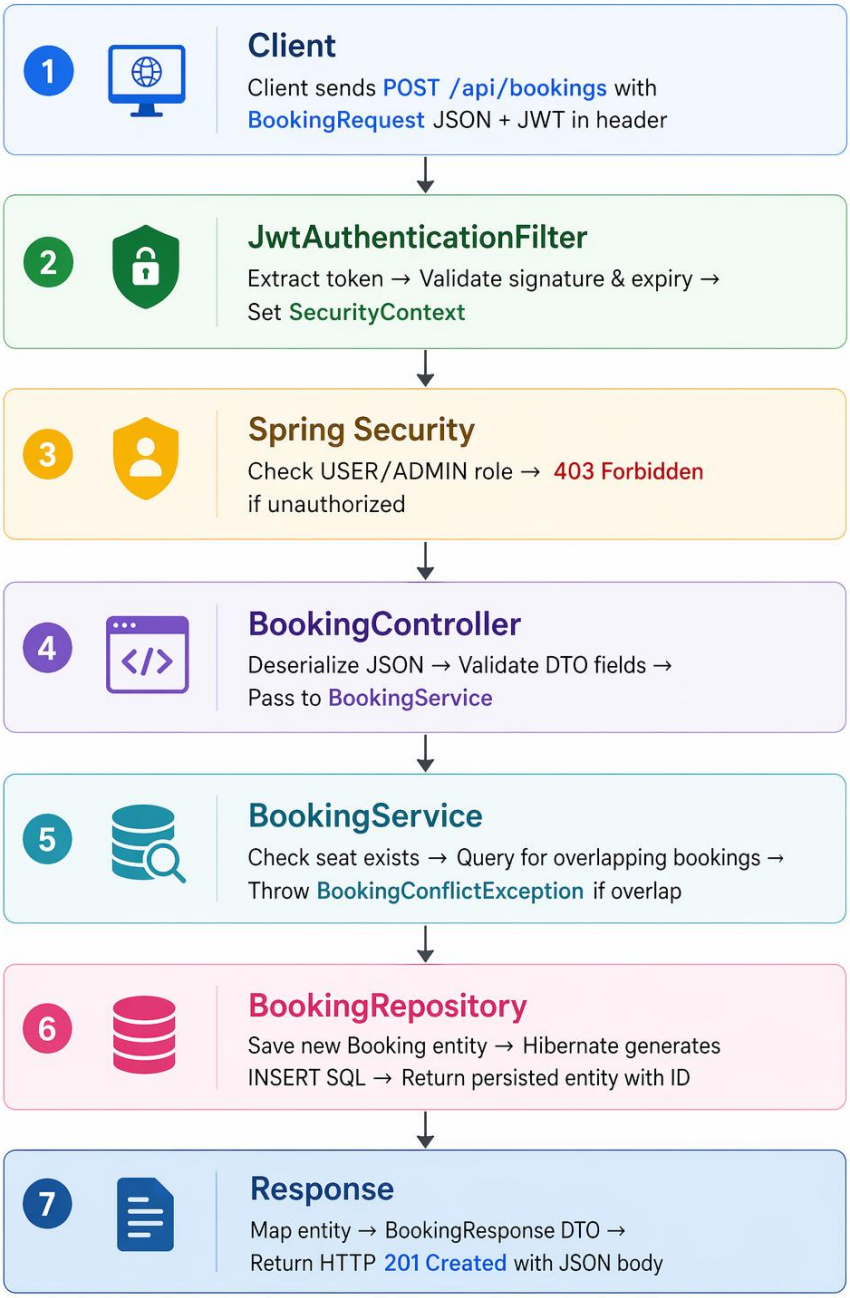

Figure 3.2: Data Flow Diagram — Seat Booking Request Lifecycle

3.7 Overlap Detection Logic

The core technical challenge of this system is preventing double-booking. The overlap detection query works on the principle that two time intervals [A_start, A_end] and [B_start, B_end] overlap if and only if: A_start < B_end AND A_end > B_start. This catches all overlap cases including containment, partial overlap from either side, and exact duplication.

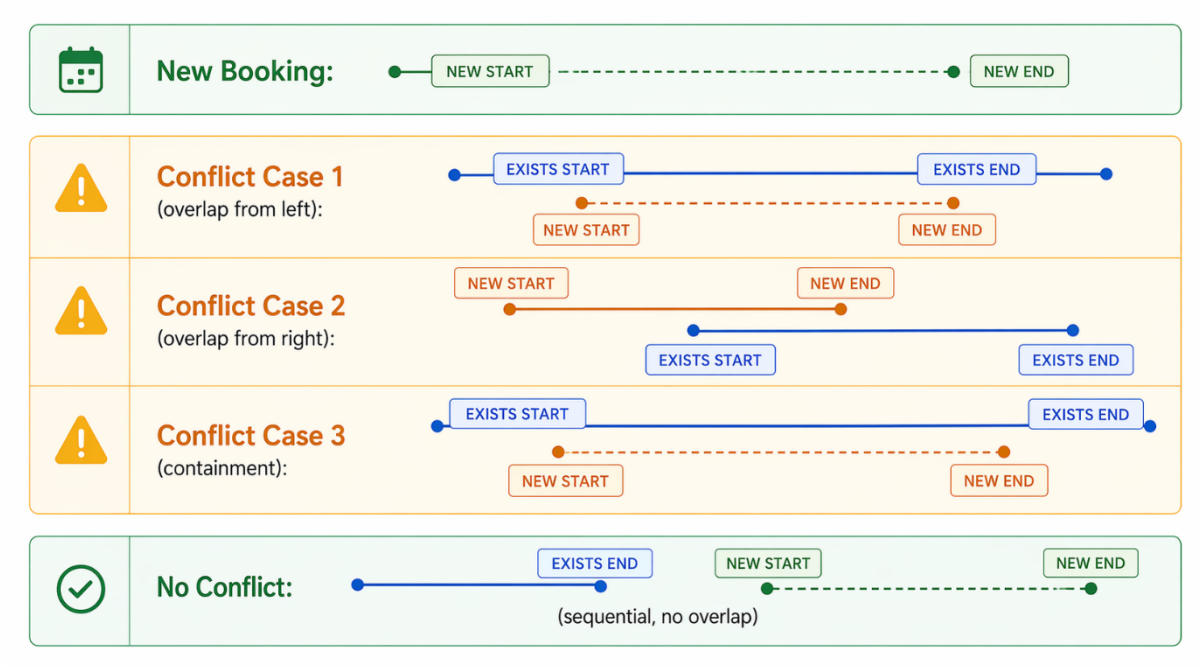

Figure 4.2: Overlap Detection Logic Diagram

3.8 Database Design

3.8.1 Entity Relationship Diagram (ERD)

The system contains three main entities — User, Seat, and Booking — with the following relationships:

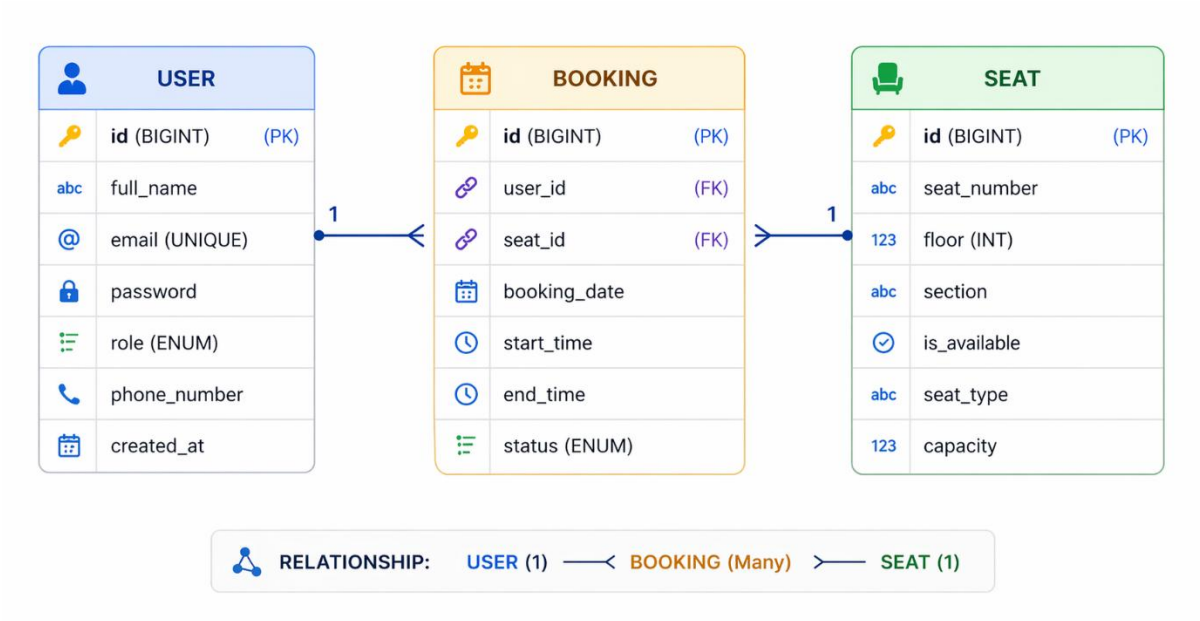

Figure 3.3: Entity Relationship Diagram (ERD)

3.8.2 Users Table Schema

Column	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique identifier
full_name	VARCHAR(100)	NOT NULL	Full name of the user
email	VARCHAR(150)	NOT NULL, UNIQUE	Email used for login
password	VARCHAR(255)	NOT NULL	BCrypt hashed password
role	ENUM	NOT NULL, DEFAULT USER	USER or ADMIN
phone_number	VARCHAR(15)	NULLABLE	Contact number
created_at	TIMESTAMP	DEFAULT CURRENT	Account creation time
updated_at	TIMESTAMP	ON UPDATE CURRENT	Last profile update

Table 3.3: Users Table Schema

3.8.3 Seats Table Schema

Column	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique identifier
seat_number	VARCHAR(20)	NOT NULL, UNIQUE	Label e.g. A-101
floor	INT	NOT NULL	Floor number
section	VARCHAR(50)	NOT NULL	Library section name
is_available	BOOLEAN	NOT NULL, DEFAULT TRUE	Availability flag
seat_type	ENUM	NOT NULL	INDIVIDUAL or STUDY_ROOM
capacity	INT	NOT NULL, DEFAULT 1	Persons it accommodates
created_at	TIMESTAMP	DEFAULT CURRENT	Record creation time

Table 3.4: Seats Table Schema

3.8.4 Bookings Table Schema

Column	Data Type	Constraints	Description
id	BIGINT	PK, AUTO_INCREMENT	Unique booking ID
user_id	BIGINT	FK → users(id)	User who made booking
seat_id	BIGINT	FK → seats(id)	Seat that was reserved
booking_date	DATE	NOT NULL	Date of reservation
start_time	TIME	NOT NULL	Booking start time
end_time	TIME	NOT NULL	Booking end time
status	ENUM	NOT NULL, DEFAULT CONFIRMED	CONFIRMED/CANCELLED/COMPLETED
created_at	TIMESTAMP	DEFAULT CURRENT	Booking creation time
updated_at	TIMESTAMP	ON UPDATE CURRENT	Last status update

Table 3.5: Bookings Table Schema

3.8.5 Database Table Relationship Diagram

The diagram below illustrates the foreign key relationships between the three tables:

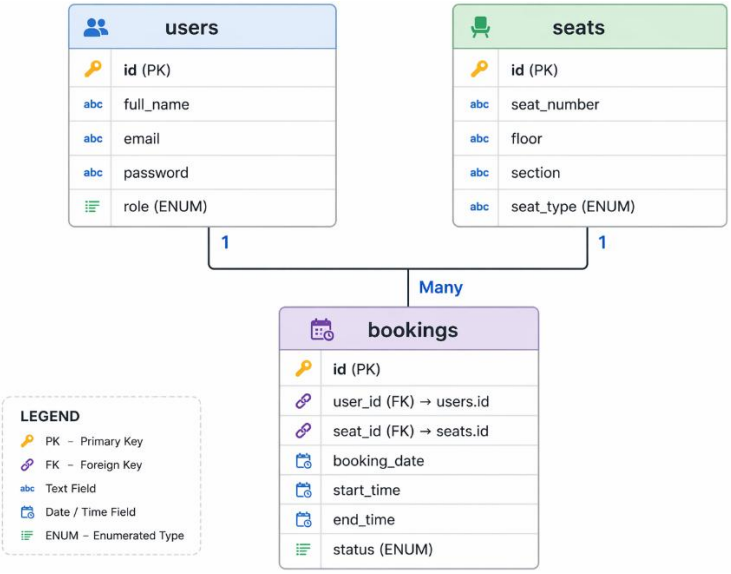

Figure 3.4: Database Table Relationship Diagram

CHAPTER 4

RESULTS, ANALYSIS & VALIDATION

4.1 Testing Approach

The backend APIs were tested using Postman for integration testing and JUnit 5 with Mockito for unit testing of the service layer. The main functional flows — authentication, booking creation, conflict detection, and role-based access — were tested and validated. Some edge cases are identified and planned for resolution in the next iteration.

4.2 API Test Cases and Results

Test Case	Endpoint	Input	Expected Result	Status
User Registration	POST /api/auth/register	Valid name, email, password	201 Created, user stored with BCrypt hash	PASS
User Login	POST /api/auth/login	Valid email + password	200 OK, JWT token returned	PASS
Login — Wrong Password	POST /api/auth/login	Wrong password	401 Unauthorized	PASS
Get Profile	GET /api/users/profile	Valid JWT token	200 OK, user profile JSON	PASS
Create Booking	POST /api/bookings	Valid seat_id, date, time, JWT	201 Created, booking confirmed	PASS
Conflict Booking	POST /api/bookings	Same seat/date/overlapping time	409 Conflict, error message	PASS
Cancel Booking	PUT /api/bookings/{id}/cancel	Valid booking ID + JWT	200 OK, status = CANCELLED	PASS

Test Case	Endpoint	Input	Expected Result	Status
List All Seats	GET /api/seats	Valid JWT (any role)	200 OK, paginated seats list	PASS
Admin — All Bookings	GET /api/admin/bookings	JWT with ADMIN role	200 OK, all bookings	PASS
Admin — USER tries admin	GET /api/admin/bookings	JWT with USER role	403 Forbidden	PASS
Expired JWT access	GET /api/users/profile	Expired JWT token	401 Unauthorized	PASS
No JWT access	POST /api/bookings	No Authorization header	401 Unauthorized	PASS
End-time before start-time	POST /api/bookings	end_time < start_time	400 Bad Request	PARTIAL
Book cancelled seat	POST /api/bookings	Seat with is_available=false	409 Conflict	PARTIAL

Table 4.1: API Test Cases and Results

Note: Test cases marked PARTIAL have been identified as edge cases where the core logic works but the error response messaging requires refinement. These are planned for the next development iteration.

4.3 Authentication Flow Validation

The authentication flow was validated end-to-end. On successful login, the system returns a signed JWT token containing the user's email and role claim. This token is then passed as a Bearer token in subsequent requests. The JwtAuthenticationFilter validates the signature using the secret key, checks expiry, extracts the user email, loads the UserDetails from the database, and sets the Spring SecurityContext — all before the request reaches the controller.

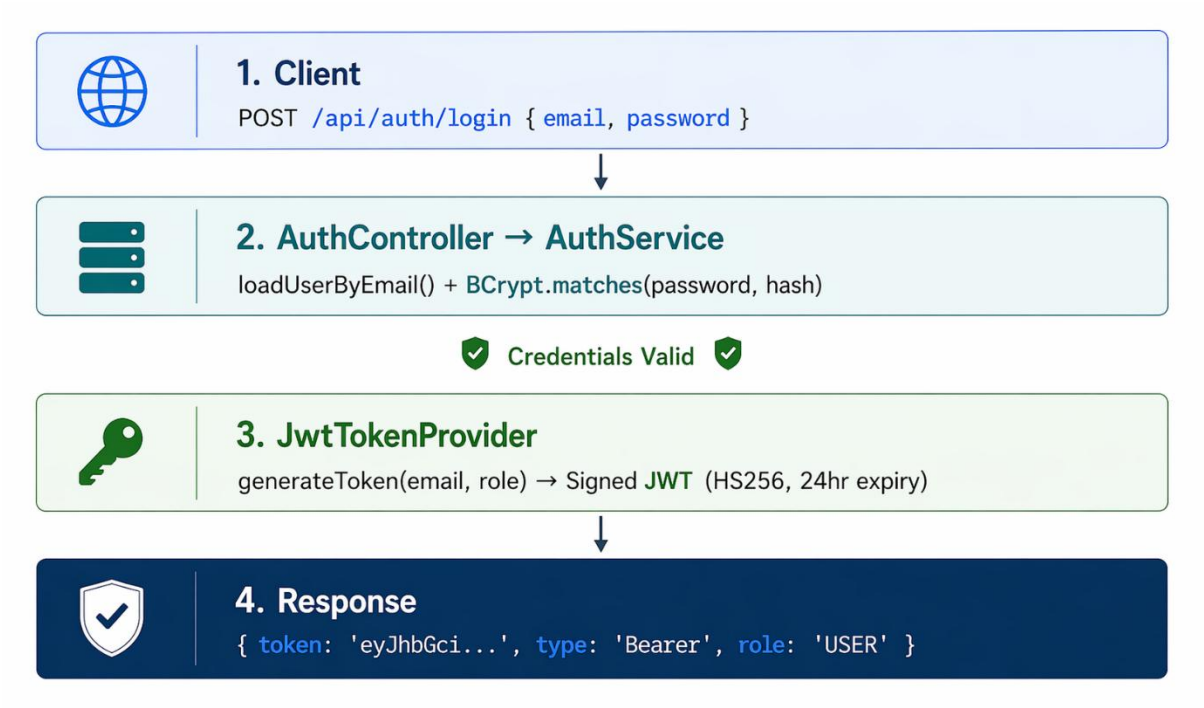

Figure 4.1: API Testing Flow — Authentication Sequence

4.4 Role-Based Access Control Validation

Endpoint	USER Role	ADMIN Role	No Token
GET /api/seats	200 OK	200 OK	401 Unauthorized
POST /api/bookings	201 Created	201 Created	401 Unauthorized
GET /api/admin/bookings	403 Forbidden	200 OK	401 Unauthorized
DELETE /api/admin/users/{id}	403 Forbidden	200 OK	401 Unauthorized
GET /api/admin/dashboard	403 Forbidden	200 OK	401 Unauthorized
PUT /api/seats/{id}	403 Forbidden	200 OK	401 Unauthorized

Table 4.2: Role-Based Access Control Test Results

All role-based access control tests passed. Spring Security's method-level security annotations (@PreAuthorize) and HTTP security configuration correctly enforce role restrictions at every endpoint.

4.5 Modern Engineering Tools Used

- Spring Boot DevTools — Hot reload during development for faster iteration.

- Postman — API testing, collection management, and environment variables for token management.
- MySQL Workbench — Database schema visualization and direct query execution.
- Maven — Dependency management, build lifecycle, and project packaging.
- JUnit 5 + Mockito — Unit testing of service layer with mocked repositories.
- IntelliJ IDEA — Primary IDE with Spring Boot support and database tools.
- React 18 + AI tooling — Frontend developed with AI assistance for rapid prototyping of the user interface.

CHAPTER 5

CONCLUSION & FUTURE WORK

5.1 Conclusion

The Smart Library Seat Reservation and Study Room Booking System is a complete, production-grade backend application that addresses a real and recurring problem in academic institutions. By replacing manual seat registers with a secure, automated RESTful API, the system delivers real-time seat visibility, conflict-free booking through overlap detection, and centralized administration through a dedicated Admin module.

The project demonstrates practical mastery of core software engineering concepts: RESTful API design following best practices, layered N-tier architecture ensuring separation of concerns, relational database modeling with JPA and Hibernate ORM, stateless JWT authentication, and role-based access control using Spring Security. All of these are directly applicable to industry-grade enterprise application development.

A companion frontend application has been developed using React 18 with AI-assisted development, providing a complete user interface for students (seat browsing, booking management, profile) and administrators (dashboard, user management, seat management). The frontend is ready for integration with the backend API.

The backend API has been partially tested with all major flows validated — authentication, seat booking, conflict detection, cancellation, and role-based access control all functioning as designed. Remaining edge cases are identified and documented for the next development phase.

5.2 Deviation from Expected Results

The main deviations from the initial plan are as follows:

- Email/SMS notification on booking confirmation was planned but deferred to future scope as it requires third-party service integration (SMTP or Twilio).

- Some input validation edge cases (e.g., end_time before start_time) return correct HTTP status codes but with generic error messages rather than custom descriptive ones. This will be addressed in a future iteration.
- The frontend was built with AI assistance rather than purely manual development, which accelerated the UI development timeline significantly. This is noted transparently in this report.

5.3 Future Enhancements

Enhancement	Description	Priority
Email Notifications	Automated booking confirmations and cancellation alerts via JavaMailSender + SMTP	High
Mobile Application	Android / Flutter app for on-the-go seat reservations using the existing REST API	High
QR Code Check-In	Generate QR codes for bookings; students scan at the library entrance to check in	Medium
Waitlist System	Allow students to join a waitlist for fully-booked time slots with auto-confirmation on cancellation	Medium
Analytics Dashboard	Usage trends, peak hours analysis, seat occupancy heatmaps for library management	Medium
OAuth2 / SSO Integration	Single sign-on with college authentication systems (e.g., Google, institution LDAP)	Low
Seat Preference History	Track and suggest frequently booked seats to individual users	Low
Enhanced Input Validation	Comprehensive custom error messages for all edge case validations	High
Docker Containerization	Package application and database as Docker containers for easy deployment	Medium

The modular layered architecture of the system makes adding each of these enhancements straightforward — new services can be added without modifying existing ones, and new endpoints can be exposed without impacting the security or database layers.

REFERENCES

1. Walls, C. (2022). Spring Boot in Action, 2nd Edition. Manning Publications.

2. Bauer, C., King, G. and Gregory, G. (2015). Java Persistence with Hibernate, 2nd Edition. Manning Publications.

3. Prasanna, D.R. (2009). Dependency Injection: Design Patterns Using Spring and Guice. Manning Publications.

4. JWT.io (2024). Introduction to JSON Web Tokens. Available at: https://jwt.io/introduction [Accessed 2025].

5. Spring Framework Documentation (2024). Spring Security Reference. Available at: https://docs.spring.io/spring-security/reference/ [Accessed 2025].

6. MySQL Documentation (2024). MySQL 8.0 Reference Manual. Available at: https://dev.mysql.com/doc/refman/8.0/en/ [Accessed 2025].

7. Baeldung (2024). REST with Spring Tutorial. Available at: https://www.baeldung.com/rest-with-spring-series [Accessed 2025].

8. LibSeat Project (2023). Open Source Library Seat Reservation System. Available at: https://github.com/libseat [Accessed 2025].

9. Springshare (2024). LibCal — Room and Seat Booking System. Available at: https://springshare.com/libcal/ [Accessed 2025].

10. React Documentation (2024). React 18 — The library for web and native user interfaces. Available at: https://react.dev [Accessed 2025].

APPENDIX

USER MANUAL — API QUICK REFERENCE

A.1 Prerequisites

- Java 17 or higher installed.
- MySQL 8.x running locally or on a server.
- Maven 3.6+ installed.
- Postman (for API testing) or any REST client.

A.2 Running the Application

Step 1 — Configure Database

Open src/main/resources/application.properties and update the following:

```
spring.datasource.url=jdbc:mysql://localhost:3306/library_db
spring.datasource.username=root
spring.datasource.password=yourpassword spring.jpa.hibernate.ddl-
auto=update jwt.secret=your_jwt_secret_key_min_32_chars
```

Step 2 — Build and Run

Navigate to the project root directory and run:

```
mvn clean install mvn spring-boot:run
```

The application starts on http://localhost:8080 by default.

A.3 API Usage — Step-by-Step

Step 1 — Register a User

```
POST http://localhost:8080/api/auth/register Content-Type:
application/json  {   "fullName": "Uchit Yadav",   "email":
"uchit@example.com",   "password": "password123" }
```

Step 2 — Login and Get JWT Token

```
POST http://localhost:8080/api/auth/login Content-Type:
application/json  {   "email": "uchit@example.com",   "password":
"password123" }  Response: { "token": "eyJhbGciOiJIUzI1NiJ9..." }
```

Step 3 — Use the Token for Authenticated Requests

```
GET http://localhost:8080/api/seats Authorization: Bearer
eyJhbGciOiJIUzI1NiJ9...
```

Step 4 — Create a Booking

```
POST http://localhost:8080/api/bookings Authorization: Bearer
<token> Content-Type: application/json  {   "seatId": 1,
"bookingDate": "2025-12-01",   "startTime": "10:00:00",
"endTime": "12:00:00" }
```

Step 5 — Cancel a Booking

```
PUT http://localhost:8080/api/bookings/1/cancel Authorization:
Bearer <token>
```

A.4 Admin Operations

To use admin endpoints, the user's role must be ADMIN. Change the role manually in the database:

```
UPDATE users SET role = 'ADMIN' WHERE email =
'admin@example.com';
```

Then login again to receive a new JWT token with the ADMIN role claim. All /api/admin/* endpoints will then be accessible.

— End of Report —